DRONE DETECTION

Full Project Documentation

Artificial Intelligence • Deep Learning • Computer Vision

Project detail	Description
Project type	Object detection using a deep-learning model
Execution platform	Google Colab
Dataset source	KaggleHub: cybersimar08/drone-detection
Detection framework	Ultralytics YOLO11n
Primary language	Python
Notebook	Untitled35(1).ipynb (uploaded notebook reviewed)

Documentation note: This report is based on the code in the supplied notebook. The notebook file does not include executed output results, so dataset counts and measured evaluation scores are intentionally left for the user to fill in after a successful run.

Contents

1 Abstract and Introduction

2 Problem Statement, Objectives, and Scope

3 Tools and Technologies

4 Dataset Description and Inspection

5 System Architecture and Workflow

6 Annotation CSV Generation

7 COCO-to-YOLO Conversion and Dataset Split

8 YOLO Configuration and Model Training

9 Evaluation Metrics

10 Inference and Prediction CSV

11 Outputs and Deliverables

12 How to Run the Notebook

13 Limitations, Risks, and Troubleshooting

14 Future Enhancements

15 Conclusion and References

1. Abstract

This project demonstrates an end-to-end drone object-detection workflow using computer vision and deep learning. The supplied Google Colab notebook downloads a dataset through KaggleHub, searches for images and COCO-style JSON annotations, creates an annotation CSV when supported annotations are available, converts bounding boxes into YOLO labels, splits records into training, validation, and test sets, trains an Ultralytics YOLO11n model, evaluates the saved best model, and performs inference on an uploaded image. Predicted object classes, confidence scores, and bounding-box coordinates are exported to a CSV file.

2. Introduction

Computer vision enables software to extract useful information from images and videos. Object detection extends image classification by identifying both the class of an object and its location using a bounding box. Drone detection is one possible application of this technology, for example in visual monitoring or research workflows. The current notebook is an experimental prototype that focuses on image-based detection; it does not implement continuous video tracking, drone control, or a deployed real-time monitoring service.

2.1 Problem Statement

Manually reviewing many images to locate objects can be slow and inconsistent. The project explores whether a pretrained object-detection model can be fine-tuned on a drone-related dataset to automatically locate annotated objects and provide structured predictions. The usefulness of the result depends on the quality and format of the dataset annotations and on the model's measured performance.

2.2 Objectives

- Download the drone-detection dataset into the Colab environment.
- Inspect available image files and annotation/configuration files.
- Identify COCO JSON annotations and export object-level ground-truth annotations to CSV.
- Convert supported bounding boxes into normalized YOLO label files.

- Create training, validation, and test folders and a YOLO YAML configuration.
- Train a pretrained YOLO11n detector with reproducible training settings.
- Evaluate the best checkpoint and report standard object-detection metrics.
- Run inference on a user-uploaded image and export the predicted boxes to CSV.

2.3 Scope

The notebook covers static-image object detection and CSV/model export. The actual class names, image count, number of bounding boxes, and final performance values must be obtained from the executed notebook. The code expects COCO-like JSON annotations for the conversion stage; other formats need a separate parser.

3. Tools and Technologies

Technology	Role in the project
Python	Main programming language for data processing and model execution.
Google Colab	Hosted notebook environment used to run the cells.
PyTorch	Deep-learning runtime and GPU availability check.
KaggleHub	Downloads the selected dataset.
Ultralytics YOLO	Loads YOLO11n, trains the detector, evaluates it, and performs inference.
Pandas	Creates annotation and prediction tables and saves CSV files.
Pillow (PIL)	Opens images and reads their dimensions.
Matplotlib	Displays sample images for visual inspection.
PyYAML	Writes the YOLO dataset configuration.
Python standard library	Path handling, JSON parsing, file copying, random splitting, and directory management.

4. Dataset Description and Inspection

The notebook calls `kagglehub.dataset_download("cybersimar08/drone-detection")` and stores the returned location in `path`. The path is wrapped as a `Path` object called `DATASET_DIR`.

4.1 Image discovery

The notebook recursively searches the dataset directory for files ending in .jpg, .jpeg, .png, .bmp, or .webp. It separately searches for .json, .xml, .txt, .csv, .yaml, and .yml files, then lists JSON and CSV files for inspection. A sample of up to nine images is displayed using Matplotlib and Pillow.

4.2 COCO annotation discovery

A JSON file is treated as a COCO annotation candidate when it is a dictionary containing the keys `images`, `annotations`, and `categories`. The notebook prints the number of images, annotations, and categories found in each matching JSON file.

4.3 Dataset facts to record after execution

Item	Actual value from successful run
Total image files found	_____
Number of JSON files	_____
Number of COCO annotation files	_____
Class names	_____
Number of annotation rows in CSV	_____
Train / validation / test image counts	_____

These values are blank because the supplied notebook has no saved execution outputs from which they can be verified.

5. System Architecture and Workflow

Stage	Operation	Main output
1. Setup	Mount Drive, inspect PyTorch and device, install packages.	Ready Colab environment
2. Download	Fetch dataset with KaggleHub.	Dataset directory
3. Inspect	Find images and annotation files; visualize sample images.	Counts and visual samples
4. Parse	Read COCO image/category/annotation structures.	Parsed records
5. CSV export	Write ground-truth bounding boxes to a CSV.	drone_annotations.csv, if supported annotations exist
6. Convert	Copy images and write normalized YOLO labels.	Train/val/test folders
7. Configure	Create YAML with paths and class names.	drone.yaml
8. Train	Fine-tune YOLO11n.	Training run and best.pt
9. Evaluate	Run validation and print detection metrics.	Precision, recall, mAP
10. Infer/export	Detect objects in uploaded image and save results.	Annotated image and drone_predictions.csv

6. Annotation CSV Generation

The annotation-export cell iterates through each COCO candidate. It creates dictionaries linking image IDs to image metadata and category IDs to category names. It then iterates through object annotations, skips crowd annotations (`iscrowd = 1`), checks that a bounding box contains four values, and ignores boxes with non-positive width or height.

6.1 CSV fields

Column	Meaning
annotation_file	Path of the source JSON file.
image_id	Image identifier in the COCO annotation.
image_file	Image filename stored in the JSON metadata.
width, height	Image dimensions stored in the annotation metadata.
class_id	Category identifier from the source annotation.
class_name	Human-readable category label.
x_min, y_min	Top-left corner of the bounding box.
x_max, y_max	Bottom-right corner calculated as x + width and y + height.
bbox_width, bbox_height	Bounding-box width and height.

The intended output is `/content/drone_annotations.csv`. This file is generated only if the notebook finds compatible COCO annotations and valid bounding boxes. The notebook also includes a cell to download this CSV from Colab.

7. COCO-to-YOLO Conversion

The dataset-preparation cell builds an image lookup by filename, reads the category list, remaps category IDs to consecutive zero-based IDs, and groups valid object annotations by image. It then shuffles the image records with a fixed random seed of 42 and divides them into train, validation, and test splits.

7.1 Split strategy

Split	Configured share	Purpose
Train	Approximately 70%	Used to update model weights.
Validation	Approximately 20%	Used to monitor and evaluate the model during development.
Test	Remaining records (approximately 10%)	Held-out data that should be used for a final evaluation if desired.

The code adjusts the split if the dataset is small and raises an error if fewer than three annotation image records are available. Actual split counts may differ slightly due to integer rounding and missing/unreadable images.

7.2 YOLO label format

YOLO object-detection labels use one line per object, typically in the form:

```
class_id x_center y_center width height
```

The class ID is zero-based. The center coordinates and box width/height are normalized by the image dimensions. The code converts the source COCO bounding box (x, y, width, height) into corner coordinates, clips the box to the image boundary, calculates the center and normalized dimensions, and writes a .txt label file.

7.3 Output folder layout

The conversion stage creates the following directory structure:

```
/content/drone_yolo/
■■■ images/train, images/val, images/test
■■■ labels/train, labels/val, labels/test
■■■ drone.yaml
```

For every image copied into a split, the code creates a corresponding label file. Images with no object annotations can have an empty label file, which represents no labeled objects. A key limitation is that the image lookup falls back to the first matching basename; if different folders contain duplicate filenames, the selected image may not be the intended one and should be checked.

8. YOLO Configuration and Model Training

The notebook creates `drone.yaml` with the dataset root, relative training and validation image paths, optional test path, and a mapping of class IDs to names. It then loads the pretrained `yolo11n.pt` model through Ultralytics and selects GPU device 0 when CUDA is available; otherwise it uses CPU.

8.1 Training parameters in the notebook

Parameter	Configured value	Explanation
Model	yolo11n.pt	Pretrained nano-sized YOLO11 detector.
Epochs	30	Maximum training passes over the training dataset.
Image size	640	Images resized/processed at 640 pixels.
Batch size	8	Images per training batch; may need reducing for limited GPU memory.
Optimizer	AdamW	Optimizer used to update model parameters.
Initial learning rate	0.001	Initial step size for optimization.
Weight decay	0.0005	Regularization setting.
Patience	8	Early stopping patience setting.
Cosine LR	True	Enables cosine learning-rate scheduling.
Random seed	42	Supports more repeatable split/training behavior.
Workers	2	Data-loading worker count.
Run directory	/content/drone_runs/drone_detection	Training output location.

The maximum is 30 epochs, but training may stop earlier if the early-stopping condition is triggered. Runtime depends on dataset size, GPU availability, and Colab resource limits. The notebook asks Ultralytics to save weights and plots.

9. Model Evaluation

After training, the notebook checks that the best checkpoint exists at `/content/drone_runs/drone_detection/weights/best.pt`, loads it, and calls the model validation routine on the validation split.

Metric	Definition	How to interpret it
Precision	TP / (TP + FP)	Of all predicted objects, the fraction that are correct. Higher precision generally means fewer false positives.
Recall	TP / (TP + FN)	Of all real annotated objects, the fraction detected. Higher recall generally means fewer missed objects.
mAP@0.5	Mean average precision at IoU = 0.50.	Measures detection quality at a relatively lenient overlap threshold.
mAP@0.5:0.95	Average mAP across IoU thresholds from 0.50 to 0.95.	A stricter summary of localization and detection quality.

TP means true positive, FP means false positive, FN means false negative, and IoU (intersection over union) measures overlap between predicted and ground-truth boxes. The notebook prints the metrics through `metrics.box.map50`, `metrics.box.map`, `metrics.box.mp`, and `metrics.box.mr`.

9.1 Results table — complete after successful training

Metric	Measured value
Precision	Not present in uploaded notebook outputs
Recall	Not present in uploaded notebook outputs
mAP@0.5	Not present in uploaded notebook outputs
mAP@0.5:0.95	Not present in uploaded notebook outputs
Epoch at best checkpoint	Record from training results
Training duration	Record from Colab run

Do not report an estimated accuracy as if it were measured. Object-detection projects are usually summarized with precision, recall, and mAP rather than classification accuracy alone. For a stronger final report, evaluate the held-out test split separately after model selection.

10. Inference on a New Image

The inference cell opens Colab's upload dialog and selects the first uploaded image. The best model predicts objects using confidence threshold 0.25 and image size 640. The result's plotted image is displayed in the notebook, and the number of detected boxes is printed.

10.1 Prediction CSV

Column	Description
image_file	Uploaded image filename/path.
class_id	Predicted class ID.
class_name	Predicted class name from the trained model.
confidence	Model confidence score for the detection.
x_min, y_min, x_max, y_max	Predicted bounding-box corner coordinates in pixels.

The prediction table is saved as `/content/drone_predictions.csv`, and the notebook includes a cell to download it. If no objects are detected, the CSV is still created with the column headers but no detection rows.

11. Outputs and Deliverables

Output	Purpose
/content/drone_annotations.csv	Ground-truth annotation table, if the CSV generation stage succeeds.
/content/drone_yolo/drone.yaml	Dataset configuration consumed by Ultralytics.
/content/drone_yolo/images/ and labels/	Prepared split images and YOLO label files.
/content/drone_runs/drone_detection/weights/best.pt	Best model checkpoint created by training.
/content/drone_predictions.csv	Predictions for the uploaded inference image.
Annotated prediction image	Visual result displayed and saved by the prediction routine.

12. How to Run the Notebook

1. Open Untitled35(1).ipynb in Google Colab.
2. Select Runtime → Change runtime type and choose a GPU if one is available.
3. Run the setup cells in order. Mount Google Drive if required by the notebook.
4. Install the required packages with the package-installation cell.
5. Download the dataset through KaggleHub and inspect the printed directory path.
6. Run the image/annotation discovery cells and review sample images.
7. Confirm that one or more valid COCO JSON files are detected before proceeding.
8. Run the annotation CSV cell if you need a ground-truth annotation spreadsheet.
9. Run the dataset-preparation cell and check that training and validation image counts are greater than zero.
10. Inspect the generated drone.yaml file and confirm class names and folder paths.
11. Load YOLO11n and start training. Wait until the training cell finishes and the best checkpoint is saved.
12. Load the best checkpoint and run validation. Record the metric values in this report.
13. Upload a sample image, run inference, and inspect the annotated output.
14. Download drone_predictions.csv and best.pt if required.

12.1 Recommended checks before training

- Training and validation image counts are non-zero.
- The dataset YAML points to the correct root directory and split paths.
- Class IDs in labels are in the range 0 to number_of_classes - 1.
- Each non-empty label line has five values: class_id, x_center, y_center, width, height.
- Normalized coordinates are within valid bounds and box width/height are positive.
- Sample images and labels visually agree.

13. Limitations, Risks, and Troubleshooting

Issue	Likely reason	Recommended check
No images found	Dataset path or file structure differs from expected.	Print DATASET_DIR and list files recursively.
No COCO annotations found	Annotations may use another format or JSON keys differ.	Inspect JSON keys and adapt the parser; do not assume COCO format.
Train/val images are zero	Image paths in JSON cannot be matched, or records are invalid.	Compare JSON file_name values with actual filenames; check missing-image count.
Class mapping mismatch	Source IDs are not contiguous or class names differ between JSON files.	Verify categories and generated YAML names.
GPU memory error	Batch size or image size is too large for available GPU.	Lower batch size or image size and rerun training.
best.pt not found	Training did not finish successfully or output path differs.	Review training logs and the actual run directory.
No detections on uploaded image	Model confidence is low, object is absent/small, or model is undertrained.	Check class mapping and image quality; test different confidence thresholds.
CSV contains no prediction rows	No detections passed the confidence threshold.	Inspect annotated image and consider threshold analysis on validation data.

13.1 Important implementation caveats

- The code searches by basename when a JSON image path cannot be resolved directly. Duplicate basenames in different folders can cause incorrect image matching.
- The notebook collects category IDs from all discovered COCO JSON files into one dictionary. If several JSON files use conflicting category IDs/names, the mapping should be reviewed.
- The code creates a random split from records rather than honoring a pre-defined dataset split. This is appropriate only if the records are suitable for random splitting and do not create leakage.
- The YAML configuration includes train and validation paths and optionally the test path. The supplied evaluation cell evaluates the validation split, not the test split.
- The uploaded notebook has no stored cell outputs; it does not prove that training succeeded or that the model achieved any particular score.

14. Future Enhancements

- Confirm the dataset's intended annotation format and build robust path matching based on relative paths rather than basename alone.
- Create a dataset audit report with image counts, class distribution, bounding-box statistics, and missing/corrupt-file checks.
- Evaluate the final selected model on the held-out test set and report per-class metrics.
- Include confusion matrix, precision-recall curves, training-loss plots, and example true-positive/false-positive/missed detections.
- Tune confidence threshold using validation data and state the chosen operating point.
- Compare YOLO model sizes and experiment with batch size, image size, augmentation, and learning rate.
- Extend image inference to video only after testing frame-by-frame detection performance.
- Package the detector in a simple user interface or API with logging, versioning, and documented limitations.

- Use human review and independent testing before any safety-critical or operational deployment.

15. Conclusion

The supplied notebook describes a complete prototype workflow for drone-related object detection using YOLO11n in Google Colab. It covers dataset download and inspection, COCO annotation parsing, annotation CSV export, YOLO label generation, random dataset splitting, model training, validation, image inference, and downloadable outputs. The next step is to run the notebook end-to-end, verify the generated dataset folders, record actual evaluation metrics, and test the model on independent images before drawing conclusions about its performance.

16. References

- Supplied project notebook: Untitled35(1).ipynb.
- KaggleHub dataset identifier used in the notebook: cybersimar08/drone-detection.
- Ultralytics YOLO training and prediction APIs are used by the notebook; consult the official Ultralytics documentation for version-specific options.
- COCO annotation structure is used as the expected source annotation format in the notebook's parsing logic.

Final submission checklist: Add your name, institution/course details if needed, run date, dataset counts, class names, training plots, sample prediction screenshots, and actual validation/test metric values before submitting.